

Design and Analysis of Algorithms

Assignment 3 & 4

Q1. Explain polynomial time reducibility with example ?

Ans. Polynomial time reducibility is a method used in computer science to compare the difficulty of two computational problems.

We say that problem A is polynomial-time reducible to problem B, written as

$A \leq_p B$

if there exists a function f that can transform any instance of A into an instance of B in polynomial time, such that:

- The answer to the instance of A is Yes if and only if the answer to the transformed instance of B is Yes.

Purpose of Polynomial Time Reducibility

1. It helps to classify problems based on their computational complexity.
2. It is used to prove NP-completeness — if an NP-complete problem can be reduced to another problem in polynomial time, that problem is also NP-complete.
3. It shows that if problem B can be solved efficiently, then problem A can also be solved efficiently.

Example :-

Consider two easy problems:

- A: Check whether a number is even.
- B: Check whether a number is divisible by 2.

To reduce A to B, simply use the same number and check divisibility by 2.

Since this check takes constant (polynomial) time, the reduction is valid.

Hence, $A \leq_p B$.

Q2. Define and differentiate:

- NP Packet Graph
- NP Scheduling Problem

Ans. NP Packet Graph :- An NP Packet Graph refers to a graph-based representation of a computational problem that belongs to the NP (Nondeterministic Polynomial time) class.

In such problems, the solution can be verified in polynomial time, even if finding that solution may take non-polynomial (exponential) time.

Example:

- Traveling Salesman Problem (TSP) — represented as a graph of cities (nodes) and paths (edges).

Finding the shortest path visiting all cities once is NP-hard, but verifying a given path's cost is polynomial.

This is an example of an NP graph problem.

NP Scheduling Problem :- An NP Scheduling Problem is a type of optimization problem where a set of tasks must be assigned to limited resources (like machines, processors, or time slots) under certain constraints, and the goal is to minimize or maximize a given objective (like time, cost, or resource usage).

These problems are in the NP class, meaning:

- It's hard (possibly exponential time) to find the best schedule.
- But once a schedule is given, it's easy to verify in polynomial time whether it satisfies all constraints.

Examples:

- Job Scheduling Problem: Assigning n jobs to m machines to minimize total completion time.
- Exam Scheduling: Assigning exams to different time slots without conflict.
- CPU Scheduling: Deciding the order of process execution to optimize waiting or turnaround time.

Q3. What is a non-deterministic algorithm? Give one example?

Ans. Non-Deterministic Algorithm :- A non-deterministic algorithm is an algorithm that, for a given input, may exhibit different behaviors on different runs — even though the input is the same.

Example:

Problem: “Does a graph have a Hamiltonian cycle?”

(A Hamiltonian cycle is a path that visits every vertex exactly once and returns to the starting point.)

Non-deterministic Algorithm Steps:

1. Guess a sequence of vertices (a possible cycle).
2. Check whether the guessed sequence forms a valid Hamiltonian cycle.

Q4. What is a state space tree? State two ways to search for an answer node ?

Ans. A State Space Tree is a conceptual tree structure used to represent all possible states or configurations that a problem can take while searching for a solution. Each node in the tree stands for a particular state of the problem, and each branch represents an action or decision that leads to another state. The root node denotes the initial state, while the leaf nodes represent the goal or final states.

This approach is commonly used in backtracking and branch-and-bound algorithms, where the tree helps visualize how different decisions lead to potential solutions or dead ends.

Two Ways to Search for an Answer Node

1. Breadth-First Search (BFS):- In this method, nodes are explored level by level. All nodes at the current depth are expanded before moving to the next level.
 - Advantages: Guarantees finding the shortest or optimal solution if all steps have equal cost.
 - Disadvantages: Requires more memory since it stores all nodes at the current level.
2. Depth-First Search (DFS):-
In this approach, one path is followed deeply until no further progress can be made, then backtracking occurs to explore other possibilities.
 - Advantages: Uses less memory and is simple to implement.
 - Disadvantages: May get stuck in long or infinite paths and does not always guarantee the shortest solution.

Q5. Explain Boolean Satisfiability (SAT) problem ?

Ans. The Boolean Satisfiability Problem (SAT) is one of the most fundamental problems in computer science and logic. It deals with determining whether there exists an assignment of truth values (True or False) to a given set of Boolean variables such that the entire logical expression (formula) evaluates to True.

Q6. Define Cook's Theorem in one line and state one application ?

Ans. Cook's Theorem states that the Boolean Satisfiability (SAT) problem is the first NP-Complete problem, meaning every problem in NP can be reduced to SAT in polynomial time.

Application :-

It is used to prove that other computational problems are NP-Complete by reducing them to the SAT problem.

Q7. Differentiate:

1. P, NP, NP-complete, NP-hard
2. Write two examples for each class (short titles only).

Ans.

Class	Meaning	Verification Time	Solving Time	Relationship	Examples (Short Titles)
P	Problems that can be solved quickly (in polynomial time) by a deterministic algorithm.	Fast	Fast	Subset of NP	1 Sorting Algorithms 2 Shortest Path
NP	Problems whose solutions can be verified quickly (in polynomial time), but may not be easy to solve.	Fast	May be Slow	Includes P	1 Traveling Salesman 2 Hamiltonian Cycle
NP-Complete	Problems that are both in NP and as hard as any NP problem (every NP	Fast	Hard	Subset of NP-Hard	1 SAT Problem 2 3-CNF Satisfiability

Class	Meaning	Verification Time	Solving Time	Relationship	Examples (Short Titles)
	problem can be reduced to them).				
NP-Hard	Problems at least as hard as NP-Complete problems, but not necessarily in NP (solution may not be verifiable in polynomial time).	May be Hard	Hard	Superset of NP-Complete	1 Halting Problem 2 Travelling Salesman (Optimization Form)

Q8. Explain the Hamiltonian Cycle problem in context of NP ?

Ans. The Hamiltonian Cycle Problem is a well-known problem in graph theory and computational complexity. It asks whether there exists a cycle in a given graph that visits each vertex exactly once and returns to the starting vertex.

Hamiltonian Cycle Problem in Context of NP:

1. The Hamiltonian Cycle Problem belongs to the NP (Nondeterministic Polynomial time) class.
2. This means that:
 - It is hard to solve directly in polynomial time.
 - However, if someone provides a proposed cycle, it can be verified quickly (in polynomial time) whether it is a valid Hamiltonian cycle.

Q9. Dijkstra's Algorithm

- State the core logic in one line.
- Write pseudocode or main function steps only.

Ans. Key Recursive Function for N-Queen Problem (with minimal comments)

```
def solveNQueens(board, row, n):
# Base case: All queens placed
if row == n:
```

```

    print(board) # or store solution
    return True

# Try placing queen in each column of current row
for col in range(n):
    if isSafe(board, row, col, n):
        board[row][col] = 1 # Place queen
        solveNQueens(board, row + 1, n) # Recurse for next row
        board[row][col] = 0 # Backtrack if not valid

return False

```

Helper Function (for clarity):

```

def isSafe(board, row, col, n):
    # Check column and diagonals for conflicts
    for i in range(row):
        if board[i][col] == 1 or \
            (col - (row - i) >= 0 and board[i][col - (row - i)] == 1) or \
            (col + (row - i) < n and board[i][col + (row - i)] == 1):
            return False
    return True

```

Q10. N-Queen Problem

- Write only the key recursive function.
- Add minimal comments for clarity.

Ans. Key Recursive Function for N-Queen Problem (with minimal comments)

```

def solveNQueens(board, row, n):
    # If all queens are placed
    if row == n:
        print(board) # Display or store solution
        return

```

```

# Try placing queen in each column
for col in range(n):
    if isSafe(board, row, col, n): # Check if position is safe
        board[row][col] = 1      # Place queen
        solveNQueens(board, row + 1, n) # Recurse for next row
        board[row][col] = 0      # Backtrack

```

Q11. Dynamic Programming – 0/1 Knapsack Problem

Weights: {3, 4, 6, 5}

Profits: {2, 3, 1, 4}

Capacity: 8

Write a DP function/pseudocode and print total profit.

Ans. Dynamic Programming Function / Pseudocode :-

```

def knapsackDP(weights, profits, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    # Build DP table
    for i in range(1, n + 1):
        for w in range(1, capacity + 1):
            if weights[i-1] <= w:
                # Include or exclude the current item
                dp[i][w] = max(profits[i-1] + dp[i-1][w - weights[i-1]], dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    # Print maximum profit
    print("Maximum Profit =", dp[n][capacity])

```

algorithm builds a DP table where each cell $dp[i][w]$ represents the maximum profit achievable using the first i items and a knapsack capacity of w .

By comparing inclusion and exclusion of each item, it ensures the optimal total profit.

Q12. Travelling Salesman Problem –

1. Branch & Bound Take nodes A, B, C, D with edge weights (example: AB=10, AC=6, AD=8, BC=5, BD=9, CD=7).

2. State essential pseudocode steps to solve this problem.

Ans. INPUT:

nodes = {A,B,C,D}

cost[u][v] given for all $u \neq v$ (symmetric assumed)

HELPERS:

function copy_matrix(M) -> deep copy of M

function reduce_matrix(M) -> (reduced_M, reduction_cost)

// subtract min of each row, then min of each column

reduction_cost = 0

for each row r in M:

 m = min(value for value in row r if value != INF)

 if m != INF and m > 0:

 subtract m from all finite entries in row r

 reduction_cost += m

for each column c in M:

 m = min(value for value in column c if value != INF)

 if m != INF and m > 0:

 subtract m from all finite entries in column c

 reduction_cost += m

return (M, reduction_cost)

DATA STRUCTURES:

Node struct:

path: list of visited nodes in order

reduced_matrix: current cost matrix with INF for forbidden edges

cost_so_far: sum of chosen edge costs

lower_bound: cost_so_far + reduction_cost from reduced_matrix

min_heap (priority queue) ordered by lower_bound

best_solution_cost = +INF

best_solution_path = null

MAIN ALGORITHM (Branch and Bound):

initialize base_matrix from cost[u][v]

(init_reduced, init_red_cost) = reduce_matrix(copy_matrix(base_matrix))

root = Node(path = [start] // choose any start, e.g., A

reduced_matrix = init_reduced,

cost_so_far = 0,

lower_bound = init_red_cost)

push root into min_heap

while min_heap not empty:

node = pop min_heap // node with smallest lower_bound

if node.lower_bound >= best_solution_cost:

continue // prune

if node.path contains all nodes:

// complete tour: add edge back to start and check cost

last = node.path.last

back_cost = cost[last][node.path[0]]

if back_cost == INF: continue

total_cost = node.cost_so_far + back_cost

```

if total_cost < best_solution_cost:
    best_solution_cost = total_cost
    best_solution_path = node.path + [node.path[0]]
    continue

// expand node: try all children by visiting an unvisited node next
last = node.path.last
for each v not in node.path:
    if node.reduced_matrix[last][v] == INF: continue
    child_matrix = copy_matrix(node.reduced_matrix)

    // forbid returning to 'last' or reusing edges that would violate path:
    // set row of 'last' to INF and column of v to INF to enforce chosen edge
    for each k: child_matrix[last][k] = INF
    for each k: child_matrix[k][v] = INF
    // forbid direct return from v to start until path complete? (optional)
    child_matrix[v][node.path[0]] = child_matrix[v][node.path[0]] // keep allowed

    // account for chosen edge cost (original matrix)
    edge_cost = cost[last][v]
    if edge_cost == INF: continue
    child_cost_so_far = node.cost_so_far + edge_cost

    // reduce and get extra reduction cost
    (reduced_child_matrix, extra_red) = reduce_matrix(child_matrix)
    child_lower_bound = child_cost_so_far + extra_red

    if child_lower_bound >= best_solution_cost: continue // prune

```

```

child = Node(path = node.path + [v],
             reduced_matrix = reduced_child_matrix,
             cost_so_far = child_cost_so_far,
             lower_bound = child_lower_bound)
push child into min_heap

```

OUTPUT best_solution_path, best_solution_cost

Q13. Floyd-Warshall Algorithm :-

1. Explain core idea in one short sentence.
2. Write the main function or pseudocode for generating the shortest path matrix.

Ans. Floyd–Warshall algorithm repeatedly updates all-pairs shortest paths by checking if a path through an intermediate vertex offers a shorter route between any two vertices.

```
function FloydWarshall(weight[1..n][1..n])
```

```
    // Initialize distance matrix
```

```
    for i = 1 to n:
```

```
        for j = 1 to n:
```

```
            dist[i][j] = weight[i][j]
```

```
    // Main triple loop
```

```
    for k = 1 to n:           // k = intermediate vertex
```

```
        for i = 1 to n:       // i = source vertex
```

```
            for j = 1 to n:    // j = destination vertex
```

```
                if dist[i][k] + dist[k][j] < dist[i][j]:
```

```
                    dist[i][j] = dist[i][k] + dist[k][j]
```

```
    return dist                // dist holds shortest path matrix
```

